

 Państwowa Akademia Nauk Stosowanych w Krynii		Kierunek Informatyka	
Instrukcja do ćwiczeń laboratoryjnych nr:	1	Nazwa przedmiotu: Programowanie w języku Kotlin	
Temat: Podstawowe elementy języka Kotlin – część 1.		Tryb studiów: stacjonarne	
		Czas trwanie ćw. 2x45 min	
Autor materiałów: dr Marcin Skuba			

1. Treści programowe:

Podstawowe elementy języka Kotlin: struktura programu, deklaracja zmiennych oraz stałych, instrukcje warunkowe, pętle, tablice, listy

2. Cel zajęć:

Celem zajęć jest zrozumienie zasad podstawowych elementów języka Kotlin w porównaniu ze znanymi już takimi jak Java czy C++.

3. Materiały dydaktyczne

I. Struktura programu i funkcja main

W Kotlinie funkcja main jest punktem wejścia. W najnowszych wersjach nie musisz już deklarować tablicy argumentów args: Array<String>, jeśli ich nie używasz.

```
// Najprostsza forma funkcji main
fun main() {
    println("Witaj w świecie Kotlin!")
}
```

II. Zmienne i Stałe

Deklaracja zmiennych to absolutny fundament Kotlin. Język ten został zaprojektowany tak, aby kod był bezpieczny i czytelny, dlatego wprowadza rygorystyczne podejście do modyfikowalności danych.

Kotlin jest językiem statycznie typowanym - oznacza to, że typ każdej zmiennej jest znany już w momencie kompilacji i nie może się zmienić w trakcie działania programu. Jeśli zadeklarujesz zmienną jako Int, nie możesz do niej później przypisać tekstu (String).

Jednak dzięki mechanizmowi **wnioskowania typów (Type Inference)**, często nie musisz tego typu wpisywać ręcznie – kompilator "domyśla się" go na podstawie przypisanej wartości.

• Stałe (**val**) vs Zmienne (**var**)

W Kotlinie kluczowym wyborem programisty jest decyzja: "Czy ta wartość będzie się zmieniać?".

val (Value) – Tylko do odczytu

Odpowiednik final w Javie lub const w innych językach. Raz przypisana wartość nie może zostać zmieniona.

Dobra praktyka mówi, iż zawsze zaczynaj od val. Używaj var tylko wtedy, gdy masz ku temu wyraźny powód.

var (Variable) – Modyfikowalna

Zmienna, której wartość możesz nadpisać w dowolnym momencie (o ile typ danych się zgadza).

• Składnia i przykłady

Schemat deklaracji wygląda następująco: **słowo_kluczowe nazwa: Typ = wartość**

```
// 1. Deklaracja z jawnym typem
val marka: String = "Toyota"

// 2. Deklaracja z wnioskowaniem typu (kompilator wie, że to Int)
val rokProdukcji = 2022

// 3. Zmienna modyfikowalna
var przebieg = 15000
przebieg = 15500 // OK - zmieniamy wartość

// przebieg = "Dużo" // BŁĄD! Nie można zmienić typu z Int na String

// 4. Stała compile-time (const val)
// Używana poza funkcjami dla wartości znanych przed uruchomieniem programu
// const val PI = 3.14
```

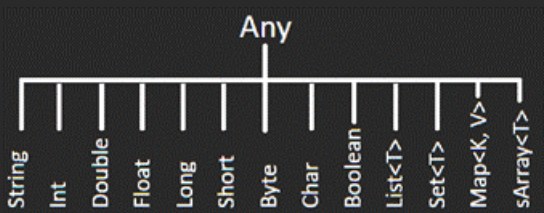
• Typy danych

Mimo że Kotlin traktuje wszystko jak obiekt (możesz wywoływać metody na liczbach), pod spodem kompilator optymalizuje je do typów prymitywnych dla wydajności.

Kategoria	Typy	Przykład
Liczby całkowite	Byte, Short, Int, Long	val id: Long = 100L
Liczby zmiennoprzecinkowe	Float, Double	val cena = 19.99
Logiczne	Boolean	var czyAktywne = true
Tekstowe	Char, String	val litera = 'A'

Typ **Any** - to fundament typów w Kotlinie.

- W Javie → Object
- W Kotlinie → **Any**



```
val text: Any = 123
print(text)    //123

val text: Any = "Tekst"
print(text)    // Tekst
```

• Bezpieczeństwo przed Nullami (**Null Safety**)

To najważniejsza cecha zmiennych w Kotlinie. Domyślnie zmienna **nie może** przechowywać wartości null.

```
var nazwisko: String = "Kowalski"
// nazwisko = null // BŁĄD KOMPILACJI

// Aby pozwolić na null, musisz dodać pytajnik przy typie:
```

```
var miasto: String? = "Warszawa"
miasto = null // Teraz jest OK
```

Podsumowanie:

- **Statyczne typowanie:** Typ jest stały i sprawdzany przy kompilacji.
- **Wnioskowanie:** Możesz pominąć typ, jeśli przypisujesz wartość od razu.
- **Niezmiennosc:** Preferuj **val** dla bezpieczeństwa kodu.
- **Null Safety:** Kotlin chroni Cię przed najczęstszym błędem programistycznym (NullPointerException).

III. Operatory w Kotlin

- Operatory matematyczne:

Operator	Znaczenie	Przykład
+	dodawanie	a + b
-	odejmowanie	a - b
*	mnożenie	a * b
/	dzielenie	a / b
%	modulo (reszta z dzielenia)	a % b
++	inkrementacja	a++
--	dekrementacja	a--

- Operatory przypisania

Operator	Znaczenie
=	przypisanie
+=	dodaj i przypisz
-=	odejmij i przypisz
*=	pomnóż i przypisz
/=	podziel i przypisz
%=	modulo i przypisz

- Operatory porównania

Operator	Znaczenie
==	równe (porównanie wartości) a nie referencje (jak w Javie)
!=	różne
>	większe
<	mniej
>=	większe lub równe
<=	mniej lub równe
===	Porównuje referencje sprawdza czy to dokładnie ten sam obiekt w pamięci
!==	różne referencje

- Podstawowe operatory logiczne (short-circuit)

Short-circuit (krótkie obliczanie) oznacza, iż Kotlin przestaje sprawdzać warunki, gdy zna już wynik.

Czyli nie zawsze wykonuje wszystkie wyrażenia logiczne.

Operator	Znaczenie
&&	AND (i)
	OR(lub)
!	NOT (negacja)

Przykład:

```
if (text != null && text.length > 3) {  
    println("OK")  
}
```

Jeśli text jest null:

- pierwszy warunek = false
- drugi NIE zostanie sprawdzony
- brak błędu NullPointerException

To ogromna zaleta.

- Operatory logiczne bez short-circuit (funkcje infix)

To NIE są specjalne operatory składniowe. To są zwykłe funkcje infix. Funkcja zawsze wykonują obie strony.

Czyli:

a and b to tak naprawdę a.and(b)

Operator	Odpowiednik
and	AND
or	OR
xor	XOR

- Operatory null-safety (specjalne dla Kotlina)

Operator	Znaczenie	Przykład	Opis
?.	bezpieczne wywołanie	<code>val text: String? = "Kotlin"</code> <code>println(text?.length) // 6</code>	Wykonaj operację tylko jeśli obiekt NIE jest nullem.
?:	Elvis operator	<code>val text: String? = null</code> <code>val length = text?.length ?: 0</code> <code>println(length) // 0</code>	Jeśli wartość po lewej jest nullem → użyj wartości po prawej.
!!	wymuszenie nie-null	<code>val text: String? = "Kotlin"</code> <code>println(text!!.length) // 6</code>	„Jestem pewien, że to NIE jest null — nie sprawdzaj.”
as?	bezpieczne rzutowanie	<code>val obj: Any = "Kotlin"</code> <code>val text = obj as? String</code> <code>println(text) // Kotlin</code>	Spróbuj rzutować. Jeśli się nie da - zwróć null.

Różnica między as a as?

- `val text = obj as String` Jeśli się nie uda - wyjątek `ClassCastException`
- `val text = obj as? String` Jeśli się nie uda - null

- Inne

Operator	Znaczenie	Przykład
..	Operator zakresu	if (x in 1..10)
in	Zawiera się	if (x in 1..10)
!in	Nie zawiera się	
is	Sprawdzanie typu	if (obj is String)
!is	Negacja typu	
*	Operator rozpakowania (spread)	val arr = intArrayOf(1,2,3) println(sum(*arr))

IV. Pobieranie danych z klawiatury

a. Standardowy sposób z Scanner

```
import java.util.Scanner

fun main() {
    val scanner = Scanner(System.`in`)

    print("Podaj swoje imię: ")
    val name = scanner.nextLine() // pobiera cały wiersz
    println("Cześć, $name!")

    print("Podaj swój wiek: ")
    val age = scanner.nextInt() // pobiera liczbę całkowitą
    println("Masz $age lat.")
}
```

nextLine() – pobiera tekst (String)
nextInt() – pobiera liczbę całkowitą (Int)-

b. Użycie readLine() – prostszy sposób w Kotlinie

```
fun main() {
    print("Podaj swoje imię: ")
    val name = readLine() // zwraca String? (nullable)
    println("Cześć, $name!")

    print("Podaj swój wiek: ")
    val age = readLine()?.toIntOrNull() // konwertuje na Int lub null jeśli
nieprawidłowy
    println("Masz $age lat.")
}
```

readLine() zwraca String? (nullable)
Trzeba uważać na null - bezpieczne rzutowanie ?. lub sprawdzanie null
toIntOrNull() – konwertuje string na Int, ale zwraca null jeśli nie da się sparsować.

Przykład z obsługą błędów:

```
fun main() {
    print("Podaj liczbę: ")
    val input = readLine()

    val number = input?.toIntOrNull()
```

```

if (number != null) {
    println("Wpisałeś $number")
} else {
    println("To nie jest poprawna liczba!")
}
}

```

To jest bezpieczny sposób w Kotlinie – nie wywali wyjątku jeśli użytkownik wpisze coś złego.

V. Instrukcje warunkowe

Podstawowa składnia instrukcji warunkowej:

```

val temperatura = 25

if (temperatura > 30) {
    println("Jest gorąco")
} else if (temperatura > 15) {
    println("Jest przyjemnie")
} else {
    println("Jest zimno")
}

```

W Kotlinie instrukcje warunkowe są znacznie potężniejsze niż w Javie, ponieważ większość z nich to **wyrażenia (expressions)**. Oznacza to, że mogą one zwracać wartość, którą przypiszesz bezpośrednio do zmiennej.

- **Przypisanie do zmiennej (Expression)**

Jeśli używasz if jako wyrażenia, gałąź else jest **obowiązkowa**.

```

val godzina = 20
val powitanie = if (godzina < 18) "Dzień dobry" else "Dobry wieczór"

// Można też użyć bloków z wieloma liniami (ostatnia linia to wartość zwracana)
val status = if (godzina > 22) {
    println("Logowanie ciszy nocnej...")
    "Noc"
} else {
    "Dzień"
}

```

W Kotlinie nie istnieje operator trójskładnikowy (znany z Javy jako **a ? b : c**). Jego rolę przejęła instrukcja **if**, która potrafi zwrócić wynik.

- **Instrukcja when (Następca switch)**

when jest jedną z najbardziej lubianych funkcji Kotliny. Jest bezpieczniejsza, bo nie wymaga słowa kluczowego break (wykonuje tylko jedną pasującą gałąź) i jest znacznie bardziej elastyczna.

```

val x = 10

when (x) {
    1 -> println("x to jeden")
    2, 3 -> println("x to dwa lub trzy") // Kilka warunków po przecinku
    in 4..10 -> println("x jest w zakresie od 4 do 10")
    !in 11..20 -> println("x jest poza zakresem 11-20")
    else -> {
        println("x to coś innego")
    }
}

```

```
}  
}
```

when bez argumentu

Możesz użyć when jako zamiennika dla długich łańcuchów if-else if. Wtedy każdy warunek musi być wyrażeniem logicznym (boolean).

```
val wynik = 85  
  
val ocena = when {  
    wynik >= 90 -> "A"  
    wynik >= 80 -> "B"  
    wynik >= 70 -> "C"  
    else -> "F"  
}
```

3. Sprawdzanie typów i Smart Casting

Kotlin potrafi automatycznie "rzutować" typ zmiennej po sprawdzeniu go w instrukcji warunkowej.

```
fun procesuj(obj: Any) {  
    when (obj) {  
        is String -> println("To tekst o długości ${obj.length}") // Automatyczne  
        rzutowanie na String  
        is Int -> println("To liczba o wartości ${obj + 10}") // Automatyczne  
        rzutowanie na Int  
        else -> println("Nieznany typ")  
    }  
}
```

Cecha	if / else	when
Złożoność	Najlepsza dla 1-2 prostych warunków.	Idealna dla wielu gałęzi i złożonych stanów.
Czytelność	Może stać się nieczytelna przy wielu else if.	Bardzo przejrzysta i zwięzła.
Wymagania	else opcjonalne (chyba że to wyrażenie).	else obowiązkowe, jeśli kompilator nie ma pewności, że wszystkie opcje są obsługiwane.

Obsługa Nulli (Elvis Operator)

```
val nick: String? = null  
val wyswietlanaNazwa = nick ?: "Gość"  
// Jeśli 'nick' jest null, przypisz "Gość"
```

VI. Pętle

Pętle w Kotlinie są zaprojektowane tak, aby były bezpieczniejsze i bardziej czytelne niż w Javie czy C++. Największą różnicą jest to, że Kotlin niemal całkowicie rezygnuje z tradycyjnej pętli for(int i=0; i<10; i++) na rzecz **zakresów (ranges)** oraz **iteracji po kolekcjach**.

- **Pętla for i zakresy (Ranges)**

Pętla for w Kotlinie służy do przechodzenia przez wszystko, co udostępnia iterator (zakresy, tablice, listy).

A. Zakresy domknięte i otwarte

Używamy operatora `..` dla zakresów obustronnie domkniętych oraz `until` dla zakresów bez ostatniego elementu.

```
// Od 1 do 5 (włącznie)
for (i in 1..5) {
    print("$i ") // Wynik: 1 2 3 4 5
}

// Od 1 do 5 (bez 5 - przydatne przy indeksach tablic)
for (i in 1 until 5) {
    print("$i ") // Wynik: 1 2 3 4
}
```

- **Krok i odliczanie w dół**

Jeśli nie chcesz zwiększać licznika o 1 lub chcesz odliczać wstecz, używasz `step` oraz `downTo`.

```
// Co drugi element
for (i in 1..10 step 2) {
    print("$i ") // Wynik: 1 3 5 7 9
}

// Odliczanie w dół
for (i in 10 downTo 1 step 3) {
    print("$i ") // Wynik: 10 7 4 1
}
```

- **Iteracja po kolekcjach (Listy i Tablice)**

Kotlin pozwala na bardzo wygodne wyciąganie elementów z list bez używania indeksów.

```
// Bezpośrednio po elementach
for (owoc in owoce) {
    println("Mamy: $owoc")
}

// Po indeksach
for (index in owoce.indices) {
    println("Owoc nr $index to ${owoce[index]}")
}

// Jednocześnie indeks i wartość (Destrukturyzacja)
for ((index, wartosc) in owoce.withIndex()) {
    println("Pozycja $index: $wartosc")
}
```

- **Pętle while oraz do-while**

Działają niemal identycznie jak w innych językach, ale warto pamiętać o jednej unikalnej cesze: zmienne zadeklarowane w warunku pętli `do-while` są widoczne w bloku `do`.

```
var energia = 3

while (energia > 0) {
    println("Pracuję...")
    energia--
}
```

```
// Do-while zawsze wykona się przynajmniej raz
do {
    val stan = "Sprawdzam system"
    println(stan)
} while (false)
```

- **Sterowanie pętlami: break, continue i Etykiety**

Podobnie jak w innych językach, break przerywa pętlę, a continue przeskakuje do następnej iteracji. Kotlin dodaje jednak **Etykiety (Labels)**, które pozwalają przerwać pętlę zewnętrzną z poziomu wewnętrznej.

```
zewnetrzna@ for (i in 1..3) {
    for (j in 1..3) {
        if (i == 2 && j == 2) break@zewnetrzna // Przerywa obie pętle na raz
        println("i:$i, j:$j")
    }
}
```

- **Funkcja forEach (Podejście funkcyjne)**

W Kotlinie bardzo często zamiast pętli for używa się funkcji wyższego rzędu forEach. Jest ona czytelniejsza przy pracy z listami.

```
val liczby = listOf(10, 20, 30)

liczby.forEach { liczba ->
    println("Liczba: $liczba")
}

// Skrócona wersja z niejawnym parametrem 'it'
liczby.forEach { println("Wartość: $it") }
```

Podsumowanie:

- **for (i in 1..10)**: Gdy znasz liczbę powtórzeń.
- **for (item in list)**: Gdy chcesz przetworzyć każdy element kolekcji.
- **list.forEach { }**: Najczęstszy wybór w nowoczesnym kodzie Kotlin.
- **while**: Gdy pętla ma trwać do spełnienia konkretnego warunku (np. gra działa dopóki `isAlive == true`).

VII. Tablice i Listy

W Kotlinie podejście do kolekcji jest jednym z najważniejszych elementów języka. Główną różnicą w stosunku do wielu innych języków jest wyraźny podział na kolekcje **modyfikowalne** (Mutable) i **niemodyfikowalne** (Immutable / Read-only).

- **Tablice (Array)**

Tablice w Kotlinie mają **stały rozmiar**. Oznacza to, że po ich utworzeniu nie możesz dodać ani usunąć elementu, ale możesz zmieniać zawartość poszczególnych komórek (jeśli nie są zablokowane).

Sposoby deklaracji:

- `arrayOf()` – tworzy tablicę z podanymi elementami.
- `Array(size) { logic }` – konstruktor pozwalający wygenerować tablicę za pomocą logiki (np. wypełnienie zerami).

- `intArrayOf()`, `doubleArrayOf()` – zoptymalizowane wersje dla typów prymitywnych (unikają narzutu pamięciowego).

```
// Prosta tablica tekstów
val kraje = arrayOf("Polska", "Niemcy", "Czechy")

// Tablica o rozmiarze 5 wypełniona potęgami liczby 2 (0, 1, 4, 9, 16)
val kwadraty = Array(5) { i -> i * i }

// Dedykowana tablica dla liczb całkowitych (zoptymalizowana)
val oceny = intArrayOf(5, 4, 3, 6, 2)

// Edycja elementu
kraje[1] = "Słowacja" // Działa

println("Liczba krajów: ${kraje.size}")
println("Pierwszy kraj: ${kraje[0]}")

// deklaracja pustych tablic 10 elementowych dla różnych typów

val liczby = IntArray(10) // [0, 0, ..., 0]
val ceny = DoubleArray(10) // [0.0, 0.0, ..., 0.0]
val wartosci = FloatArray(10) // [0.0, 0.0, ..., 0.0]
val dystans = LongArray(10) // [0, 0, ..., 0]
val litery = CharArray(10)
// Klasa String nie jest typem prymitywnym, więc nie ma "StringArray". Musimy użyć
// ogólnego konstruktora Array i podać wartość początkową (np. pusty ciąg znaków).
val teksty = Array(10) { "" } // [ "", "", ..., "" ]
val uzytkownicy = Array(10) { User() }
// Jeśli chcesz stworzyć tablicę, która na początku jest "naprawdę pusta" (nie ma w niej
// nawet zer ani pustych stringów), musisz pozwolić na przechowywanie wartości null.
val pustaTablica = arrayOfNulls<String>(10) // [null, null, ..., null]
val pusteLiczby = arrayOfNulls<Int>(10) // [null, null, ..., null]
```

• Listy Tylko-Do-Odczytu (List)

Domyślna lista w Kotlinie jest **niemodyfikowalna**. Nie posiada metod takich jak `.add()` czy `.remove()`. Jest to bezpieczne i zalecane w programowaniu funkcyjnym.

```
// Deklaracja listy (Read-only)
val dniTygodnia = listOf("Poniedziałek", "Wtorek", "Środa")

// dniTygodnia.add("Czwartek") // BŁĄD KOMPILACJI - nie ma takiej metody!

println(dniTygodnia.last()) // Pobranie ostatniego elementu
println(dniTygodnia.contains("Sobota")) // Zwróci false
```

• Listy Modyfikowalne (MutableList)

Jeśli planujesz dynamicznie zarządzać zawartością listy (np. dodawać produkty do koszyka), musisz użyć `mutableListOf()`. Jest to odpowiednik `ArrayList` z Javy.

```
// Deklaracja pustej listy modyfikowalnej (wymaga podania typu w <>)
val listaZakupow = mutableListOf<String>()

// Dodawanie elementów
listaZakupow.add("Chleb")
```

```
listaZakupow.add("Mleko")
listaZakupow.addAll(listOf("Masło", "Jajka"))

// Usuwanie elementów
listaZakupow.remove("Mleko")
listaZakupow.removeAt(0) // Usuwa pierwszy element (Chleb)

// Podmiana elementu
listaZakupow[0] = "Ser"

println("Twoja lista: $listaZakupow")
```

Cecha	Array (Tablica)	List (Lista niemodyfikowalna)	MutableList (Lista modyfikowalna)
Rozmiar	Stały (niezmienny)	Stały (niezmienny)	Dynamiczny (można dodawać/usuwać)
Zawartość	Można zmieniać (arr[i] = x)	Nie można zmieniać	Można zmieniać
Wydajność	Najszybsza, niskopoziomowa	Bardzo bezpieczna	Elastyczna, ale nieco wolniejsza
Kiedy użyć?	Gdy znasz dokładną liczbę elementów i liczy się wydajność.	Domyślnie dla wszystkich stałych zestawów danych.	Gdy lista będzie rosła lub malała w czasie działania programu.

Przykład wykorzystania **foreach** oraz **when**:

```
var oceny = intArrayOf(3,5,6,2,4,3,1)

oceny.forEach { ocena ->
    val wynik = when {
        ocena >= 1 && ocena < 3 -> "nie zaliczone"
        ocena >= 3 && ocena <= 6 -> "zaliczone"
        else -> "nieprawidłowa ocena"
    }
    println("Ocena $ocena - wynik $wynik")
}

// obliczanie średniej za pomocą funkcji wyższego rzędu
var srednia = oceny.average()

// Zaokrąglenie do określonej liczby miejsc po przecinku
var srednia_zaokraglona = "%.2f".format(srednia)
println("Średnia ocen: $srednia_zaokraglona")
```

VIII. Zadania

Zadanie 1: Pole powierzchni kuli

Napisz program o nazwie „Pole” obliczający pole powierzchni kuli. Wartość promienia wpisz z klawiatury.

Zadanie 2 – Kalkulator prosty z wyrażeniami warunkowym.

Napisz program, który pobiera od użytkownika dwie liczby (a i b) oraz znak działania (+, -, *, /). Program powinien obliczyć wynik i wypisać go na ekranie, używając instrukcji if zwracającej wyrażenie.

Zadanie 3 – Ocena na podstawie punktów z when

Napisz program, który pobiera od użytkownika liczbę punktów zdobytych na egzaminie (0–100) i przypisuje ocenę według poniższej skali, używając **when** jako wyrażenia:

Punkty Ocena

90–100 - 5

75–89 - 4

60–74 - 3

50–59 - 2

0–49 - 1

when powinno zwracać ocenę jako wynik wyrażenia. Obsłuż sytuację, gdy użytkownik wpisze liczbę spoza zakresu 0–100

Zadanie 4 – TABLICE - Analiza wyników testu

Napisz program, który:

1. Tworzy tablicę do przechowywania liczb całkowitych o rozmiarze 5.
2. Pobiera od użytkownika 5 wyników punktowych (0–100) i zapisuje je do tablicy.
 - Za pomocą pętli for (i in indices) oblicza sumę punktów
 - Za pomocą forEach wypisze wszystkie wyniki
3. Oblicza i wypisuje średnią.
4. Znajduje największy wynik (używając klasycznej pętli for).

W zadaniu można wykorzystać metody rozszerzające np.:

```
val x: Int = 5
val y: Double = x.toDouble()
```

Zadanie 5 – LISTA NIEMUTOWALNA (List) - Analiza temperatur

Napisz program, który:

1. Tworzy niemutowalną listę temperatur: **temperatures** z wartościami: **12, 18, 25, 30, 15, 22**
2. Za pomocą pętli for policzy ile dni miało temperaturę powyżej 20°C
3. Za pomocą **forEach** wypisze komunikat:
 - „Gorąco” jeśli > 25
 - „Umiarkowanie” jeśli 15–25
 - „Zimno” jeśli < 15
4. Obliczy średnią temperaturę.

Zadanie 6 – LISTA MUTOWALNA (MutableList) - Zarządzanie listą zakupów

Napisz program, który:

1. Tworzy pustą MutableList<String>.
2. W pętli for (np. 5 razy):
 - pobiera nazwę produktu z klawiatury
 - dodaje go do listy
3. Wypisuje wszystkie produkty używając forEach.

4. Usuwa jeden produkt podany przez użytkownika.
5. Ponownie wypisuje listę przy użyciu klasycznej pętli for z indeksami.
6. Na końcu wypisuje liczbę produktów na liście.